

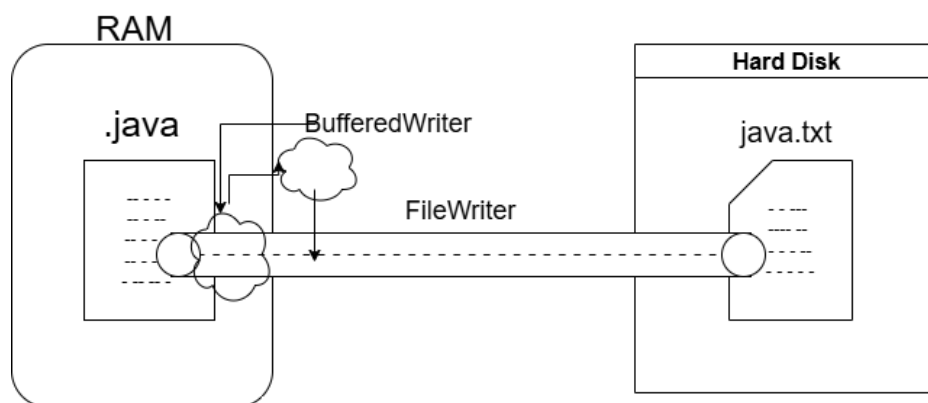
BufferedWriter and FileWriter

When we use just **FileWriter** to write data to a file, each character gets written individually to the hard disk. This means if we need to write 100 characters, the program will make 100 separate "trips" (or disk operations) to the hard disk. This is inefficient because:

- Each disk operation has overhead
- Multiple small operations are slower than fewer larger operations
- Hard disk access is much slower than memory operations

👉 **BufferedWriter**

- **BufferedWriter** solves this performance issue by creating an in-memory buffer that temporarily stores the data before writing it to disk.



In the diagram:

- A temporary buffer (memory area) is created in RAM
- Data is first written to this buffer instead of directly to the disk
- When the buffer fills up or when we flush/close the writer, all data is written to the disk at once
- This significantly reduces the number of disk operations

👉 **The Water Pipe Analogy**

- Imagine you need to transfer water from one location to another through a pipe:

🔧 **FileWriter alone:** Like manually pouring water cup by cup through the pipe

🔧 **BufferedWriter + FileWriter:** Like attaching an electric motor pump that collects water in a tank and then pushes it through the pipe in larger batches

This pumping system (BufferedWriter) makes the whole process much faster and more efficient.

Example:

```
import java.io.*;
public class LaunchIO {
    public static void main(String[] args) throws IOException{
        String filePath="D:\\telusko\\InputOutput\\java.txt";
        FileWriter fileWriter=null;
        BufferedWriter bufferedWriter=null;

        try{
            // Creating a File object that represents the physical file
            File file1=new File(filePath);

            // Creating a FileWriter - the basic stream to write to file
            // 'true' means append mode - will add to existing file content
            fileWriter=new FileWriter(file1,true);

            // Attaching BufferedWriter to make writing more efficient
            bufferedWriter=new BufferedWriter(fileWriter);

            // Writing a string "Alien"
            bufferedWriter.write("Alien");

            // Adding a new line
            bufferedWriter.newLine();

            // Writing character with ASCII value 66 (which is 'B')
            bufferedWriter.write(66);
            bufferedWriter.newLine();

            // Writing character with ASCII value 97 (which is 'a')
            bufferedWriter.write(97);
            bufferedWriter.newLine();

            // Writing an array of characters
            char[] ch={'j','a','v','a'};
            bufferedWriter.write(ch);

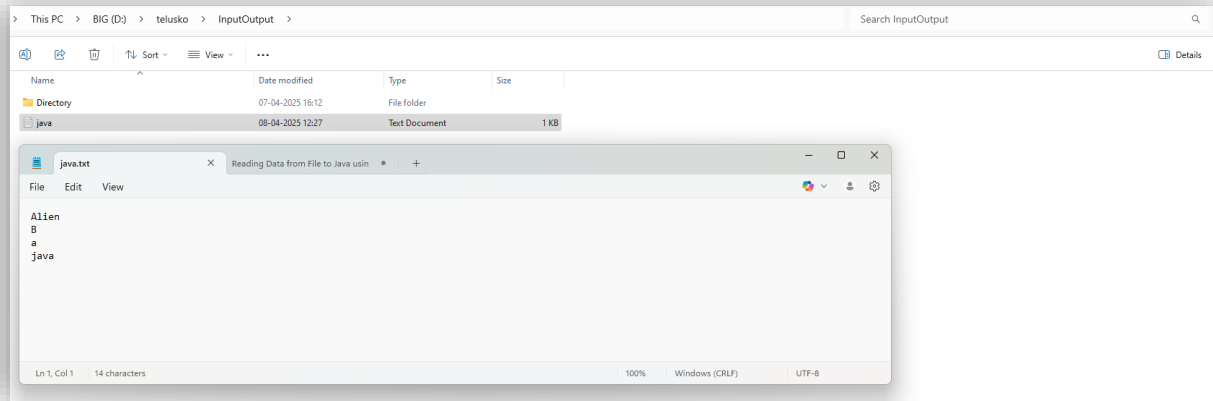
            System.out.println("Open java.txt file in your hd");

        }catch(Exception e){
            System.out.println("Some problem");
        }finally{
            // Flush ensures any data still in buffer is written to file
            bufferedWriter.flush();

            // Close the stream
            bufferedWriter.close();
        }
    }
}
```

Output:

Open java.txt file in your hd



👉 Key Points

- **FileWriter** alone writes character-by-character to the disk (inefficient)
- **BufferedWriter** creates a temporary storage buffer in memory
- Data is collected in the buffer and then written to the disk in larger chunks
- This significantly reduces the number of disk operations
- The **flush()** method forces any buffered data to be written to the disk
- Always close your streams with **close()** in a finally block to ensure proper resource cleanup